Módulo 2 - Grupo: JOULIE FARADAY

2026-04-15

Introducción

Este es un documento en R Markdown. Permite generar HTML o Word combinando texto y código. El lenguaje R es una poderosa herramienta matemática y estadística. En este reporte generado con R Markdown, analizaremos la resolución de distintos problemas y evaluaremos la calidad de diferentes algoritmos implementados.

Probando primeros comandos de R

R es un leguaje muy parecido a matlab y trabaja con variables que en general pueden ser matrices.

```
A <- 38
A <- 40
B <- 60
C <- B - A
C
```

```
## [1] 20
```

Uso de data sets o base de datos internos de R

Usando el comando *data()* en la consola obtenemos distintos datos ya cargados en la base de datos de R, estos datos estan cargados en tablas de las cuales podemos elegir columnas especificas si escribimos el signo *$* despues del comando de informacion especifico que queremos.

También está el comando *summary()*, que es una de las herramientas más útiles y rápidas que tiene R para hacer estadística descriptiva básica.

```
# Planteamos el dataset
data(cars)
```

```
# Usamos el símbolo $ para extraer SOLO la columna de velocidades (speed)
velocidades <- cars$speed

# Mostramos los primeros 6 valores de esa columna aislada
head(velocidades)
```

```
## [1] 4 4 7 7 8 9
```

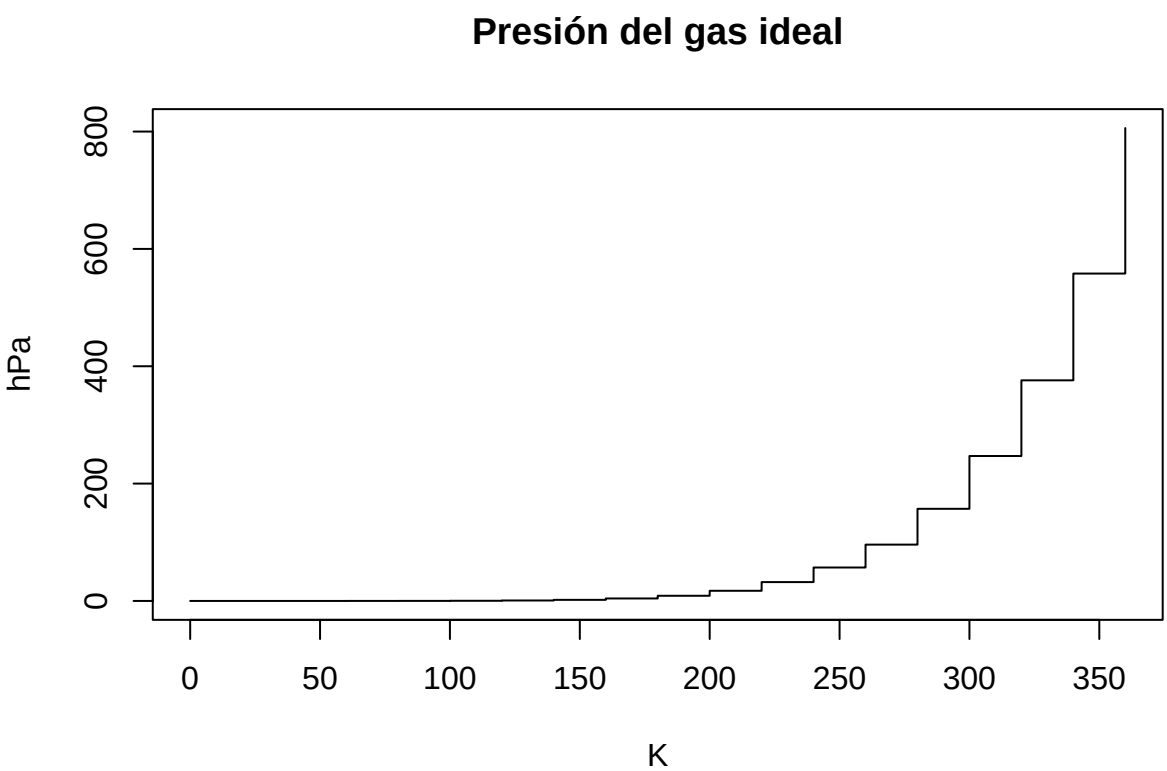
```
summary(cars)
```

```
##      speed           dist
##  Min.   : 4.0   Min.   :  2.00
##  1st Qu.:12.0   1st Qu.: 26.00
##  Median :15.0   Median : 36.00
##  Mean   :15.4   Mean   : 42.98
##  3rd Qu.:19.0   3rd Qu.: 56.00
##  Max.   :25.0   Max.   :120.00
```

Comando plot (Gráficos)

Es un comando que sirve para graficar cualquier fuente de datos que le asignemos: Tenemos la posibilidad de asignar nombres a las variables o hacer cambios del grafico, en caso de no saber el comando podemos escribir **plot** en la consola, a la derecha nos va a abrir una pestaña y ahí tocar Generic X-Y Plotting y ahi nos va a explicar cada comando

```
plot(pressure, type = "s",
     main = "Presión del gas ideal",
     ylab = "hPa",
     xlab = "K")
```



Funciones estadísticas

Otro comando usado es el comando *rnorm*. Este lo que hace es generar datos aleatorios, pero forzando a que esos datos sigan la forma de una Distribución Normal (la famosa campana de Gauss).

Podemos asignarle ciertos parámetros si queremos: *Tamaño de la muestra* **n** (es la cantidad de datos que le pides a R que invente, en este caso 350) *Media* (el valor central, en este caso 22), o *Desviación estándar* (Representa la dispersión o el "error experimental", en este caso, 5).

```
z1 <- rnorm(350, 22, 5)
z1
```

```
##   [1] 26.657276 26.341930 22.337111 10.051986 25.335628 18.299888 27.338767
##   [8] 17.183952 22.461660 21.855279 27.145947 21.459671 18.397125 19.308537
##  [15] 23.005697 25.839043 33.120667 16.877913 25.409571 21.329951 25.078074
##  [22] 20.634398 24.180488 21.557754 19.928052 24.322059 21.836065 30.749157
##  [29] 21.017194 16.908532 17.444526 33.604959 23.732936 16.135300 23.340243
##  [36] 16.696765 28.533665 26.009855 17.871337 24.592634 20.470552 22.739090
##  [43] 15.446737 21.785331 23.745543 23.070664 29.777893 17.307728 15.274548
##  [50] 26.554206 20.006086 21.179037 26.107735  8.981179 17.544916 25.392688
```

```
##  [57] 15.826073 16.935841 25.026702 26.948731 19.506616 21.997485 25.159589
##  [64] 23.115253 22.814668 25.412685 29.225901 19.172646 24.607507 13.143357
##  [71] 19.882495 22.832216 31.634564 16.221080 31.347340 31.979397 23.102774
##  [78] 26.825254 23.601458 17.421228 21.293082 24.716929 21.932595 16.571342
##  [85] 21.865678 17.982039 21.748114 23.694494 31.439586 18.570937 27.078713
##  [92] 25.365661 17.522994 24.627565 20.740871 14.073709 22.604558 22.156259
##  [99] 21.638514 28.769012 22.034033 24.387309 26.009710 23.019461 19.531064
## [106] 28.023015 24.665742 18.183700 23.173468 19.474734 21.119811 26.071241
## [113] 23.517545 14.024586 19.379936 15.519124 27.875366 27.537483 17.528571
## [120] 26.838537 20.144789 27.547382 26.680891 26.685787 24.469037 26.660025
## [127] 19.781554 17.613276 30.684046 20.025551 18.979620 20.080473 21.741839
## [134] 19.933410 23.315746 24.102130 23.753778 27.126741 20.104982 21.141450
## [141] 25.789674 19.434699 26.148512 22.805189 24.766488 27.631195 24.086847
## [148] 19.434395 18.669270 27.921144 27.016885 14.852964 27.388586 26.949312
## [155] 25.075272 15.887284 18.050426 28.232860 23.500808 30.743173 21.299364
## [162] 25.888569 25.005519 26.324868 18.119564  8.187076 26.406920 25.643669
## [169] 21.767062 18.511556 23.807127 25.024824 17.715107 18.884374 23.101365
## [176] 18.253247 22.559407 25.916183 26.362830 31.015458 23.667461 28.937231
## [183] 30.307895 28.817326 18.215108 19.389466 15.880260 26.637134 20.775295
## [190] 27.568097 23.582034 25.494055 18.959119 23.524254 24.041292 14.808457
## [197] 23.878006 22.015350 15.318311 22.995906 14.288750 24.034920 26.836690
## [204] 19.324429 13.985118 16.013889 19.036330 15.702821 15.999036 30.282272
## [211] 28.098314 18.427173 20.729895 25.097359 22.791175 21.084837 28.937273
## [218] 20.426781 18.701146 21.460596 25.159841 17.796048 16.796492 21.301657
## [225] 27.834597 22.282511 11.364307 22.368619 28.350829 20.198297 20.537154
## [232] 28.163082 20.839816 15.249748 22.442467 21.347189 17.591153 20.748339
## [239] 14.564461 23.015792 18.050256 26.434432 23.330162 16.353949 19.787475
## [246] 30.416459 28.044391 16.400757 21.190076 22.479997 23.300124 23.552674
## [253] 12.035714 27.614070 27.235809 17.612479 18.698823 31.292522 25.400275
## [260] 31.431525 32.051128 27.729516 20.406963 24.115240 15.255036 18.678209
## [267] 30.736672 23.788828 34.791708 19.354246 18.356921 22.862684 13.212565
## [274] 26.336999 19.482991 19.046733 18.231870 22.977765 16.964275 23.206276
## [281] 25.075559 27.072238 13.258374 29.689261 16.233533 28.174760 20.446432
## [288] 28.279740 25.047649 29.424154 35.007871 18.903964 26.720776 14.054143
## [295] 18.434444 20.482022 18.710756 23.604665 26.934564 16.117775 22.403669
## [302] 20.589254 20.438616 25.160549 23.710609 21.263593 27.201081 18.636905
## [309] 21.875093 26.939254 15.045315 12.588389 16.909245 23.497494 19.963308
## [316] 15.747925 16.270553 27.300299 23.831646 19.070660 11.541559 26.622698
## [323] 23.073899 27.944351 13.937840 26.785397 21.058826 25.655752 26.055959
## [330] 25.637987 34.798258 26.752733 26.201986 25.612079 14.728634 27.724032
## [337] 19.063694 14.659099 16.222319 15.557595 29.421406 18.752031 21.336061
## [344] 23.052082 30.100887 18.789009 23.191492 27.756714 20.838285 11.052346
```

```r
w1 <- length(z1)
w1
```

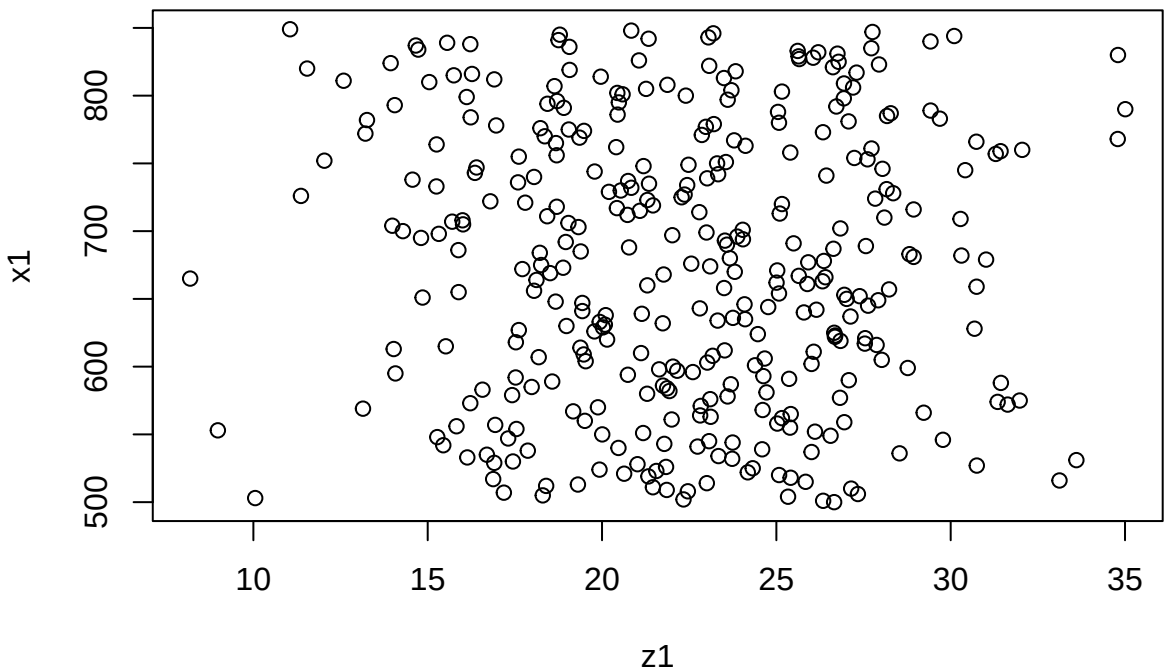
```
## [1] 350
```

```r
x1 <- 500:849
x1
```

```
##   [1] 500 501 502 503 504 505 506 507 508 509 510 511 512 513 514 515 516 517
##  [19] 518 519 520 521 522 523 524 525 526 527 528 529 530 531 532 533 534 535
##  [37] 536 537 538 539 540 541 542 543 544 545 546 547 548 549 550 551 552 553
##  [55] 554 555 556 557 558 559 560 561 562 563 564 565 566 567 568 569 570 571
##  [73] 572 573 574 575 576 577 578 579 580 581 582 583 584 585 586 587 588 589
```

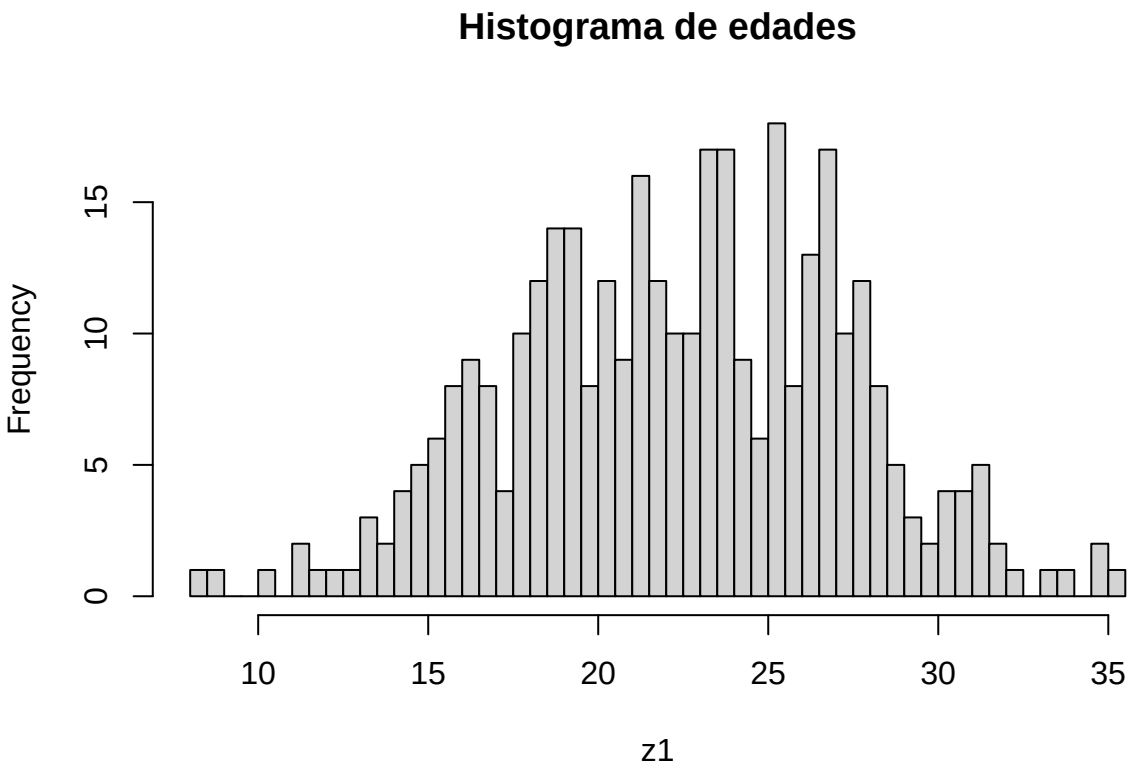
```
##  [91] 590 591 592 593 594 595 596 597 598 599 600 601 602 603 604 605 606 607
## [109] 608 609 610 611 612 613 614 615 616 617 618 619 620 621 622 623 624 625
## [127] 626 627 628 629 630 631 632 633 634 635 636 637 638 639 640 641 642 643
## [145] 644 645 646 647 648 649 650 651 652 653 654 655 656 657 658 659 660 661
## [163] 662 663 664 665 666 667 668 669 670 671 672 673 674 675 676 677 678 679
## [181] 680 681 682 683 684 685 686 687 688 689 690 691 692 693 694 695 696 697
## [199] 698 699 700 701 702 703 704 705 706 707 708 709 710 711 712 713 714 715
## [217] 716 717 718 719 720 721 722 723 724 725 726 727 728 729 730 731 732 733
## [235] 734 735 736 737 738 739 740 741 742 743 744 745 746 747 748 749 750 751
## [253] 752 753 754 755 756 757 758 759 760 761 762 763 764 765 766 767 768 769
## [271] 770 771 772 773 774 775 776 777 778 779 780 781 782 783 784 785 786 787
## [289] 788 789 790 791 792 793 794 795 796 797 798 799 800 801 802 803 804 805
## [307] 806 807 808 809 810 811 812 813 814 815 816 817 818 819 820 821 822 823
## [325] 824 825 826 827 828 829 830 831 832 833 834 835 836 837 838 839 840 841
## [343] 842 843 844 845 846 847 848 849
```

```r
plot(z1, x1)
```



El comando *hist()* genera un histograma.

```r
hist(z1, main = "Histograma de edades", breaks = 60)
```



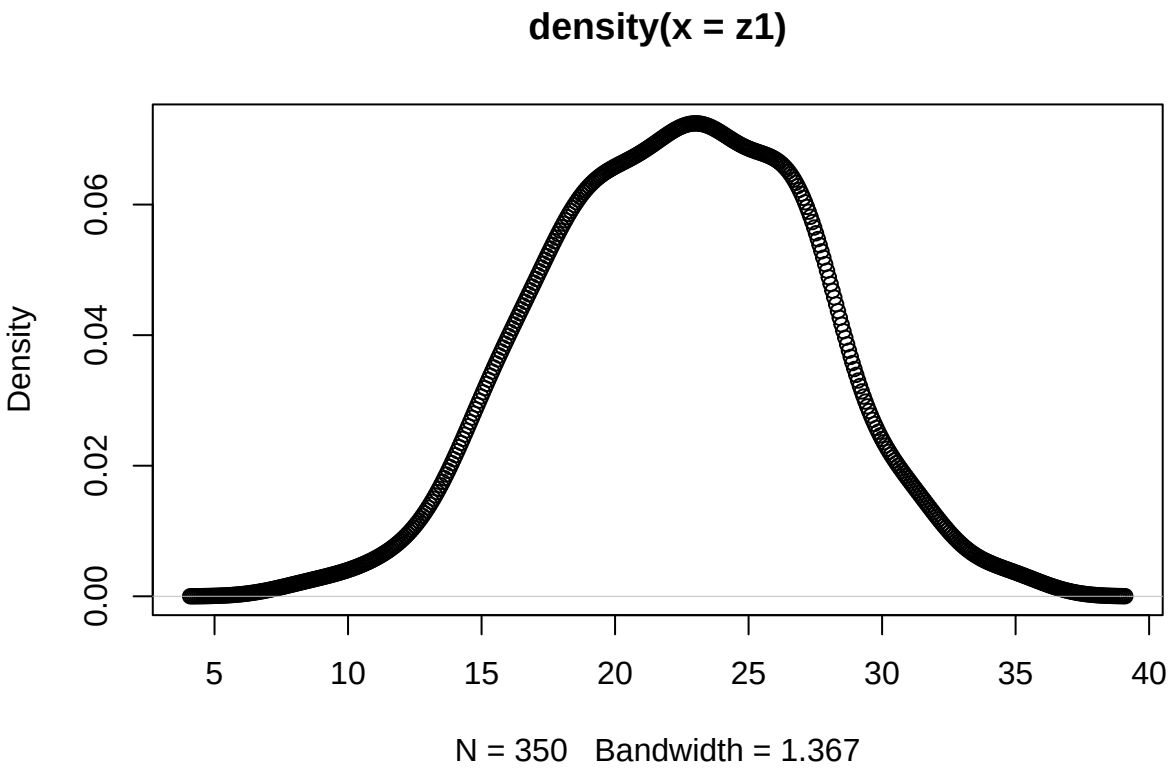
El comando *density()* calcula la densidad de probabilidad. Es la versión matemática, suavizada y continua del histograma.

```
density(z1,type="b")
```

```
## Warning: In density.default(z1, type = "b") :
##  extra argument 'type' will be disregarded
```

```
##
## Call:
##  density.default(x = z1, type = "b")
##
## Data: z1 (350 obs.); Bandwidth 'bw' = 1.367
##
##        x                y
##  Min.   : 4.088   Min.   :1.077e-05
##  1st Qu.:12.843   1st Qu.:3.009e-03
##  Median :21.597   Median :1.743e-02
##  Mean   :21.597   Mean   :2.850e-02
##  3rd Qu.:30.352   3rd Qu.:5.953e-02
##  Max.   :39.107   Max.   :7.244e-02
```

```
plot(density(z1),type="b")
```



Ejercicio 1: Designación

Consigna: Las ultimas 3 cifras del DNI son 803, crear una variable que tenga ese número

```
DNI <- 803
```

Consigna: Crear un vector del 1 al 803.

```
secuencia_dni <- 1:803
secuencia_dni
```

```
##   [1]   1   2   3   4   5   6   7   8   9  10  11  12  13  14  15  16  17  18
##  [19]  19  20  21  22  23  24  25  26  27  28  29  30  31  32  33  34  35  36
##  [37]  37  38  39  40  41  42  43  44  45  46  47  48  49  50  51  52  53  54
##  [55]  55  56  57  58  59  60  61  62  63  64  65  66  67  68  69  70  71  72
##  [73]  73  74  75  76  77  78  79  80  81  82  83  84  85  86  87  88  89  90
##  [91]  91  92  93  94  95  96  97  98  99 100 101 102 103 104 105 106 107 108
## [109] 109 110 111 112 113 114 115 116 117 118 119 120 121 122 123 124 125 126
## [127] 127 128 129 130 131 132 133 134 135 136 137 138 139 140 141 142 143 144
## [145] 145 146 147 148 149 150 151 152 153 154 155 156 157 158 159 160 161 162
## [163] 163 164 165 166 167 168 169 170 171 172 173 174 175 176 177 178 179 180
## [181] 181 182 183 184 185 186 187 188 189 190 191 192 193 194 195 196 197 198
## [199] 199 200 201 202 203 204 205 206 207 208 209 210 211 212 213 214 215 216
## [217] 217 218 219 220 221 222 223 224 225 226 227 228 229 230 231 232 233 234
## [235] 235 236 237 238 239 240 241 242 243 244 245 246 247 248 249 250 251 252
## [253] 253 254 255 256 257 258 259 260 261 262 263 264 265 266 267 268 269 270
## [271] 271 272 273 274 275 276 277 278 279 280 281 282 283 284 285 286 287 288
## [289] 289 290 291 292 293 294 295 296 297 298 299 300 301 302 303 304 305 306
## [307] 307 308 309 310 311 312 313 314 315 316 317 318 319 320 321 322 323 324
## [325] 325 326 327 328 329 330 331 332 333 334 335 336 337 338 339 340 341 342
```

```
## [343] 343 344 345 346 347 348 349 350 351 352 353 354 355 356 357 358 359 360
## [361] 361 362 363 364 365 366 367 368 369 370 371 372 373 374 375 376 377 378
## [379] 379 380 381 382 383 384 385 386 387 388 389 390 391 392 393 394 395 396
## [397] 397 398 399 400 401 402 403 404 405 406 407 408 409 410 411 412 413 414
## [415] 415 416 417 418 419 420 421 422 423 424 425 426 427 428 429 430 431 432
## [433] 433 434 435 436 437 438 439 440 441 442 443 444 445 446 447 448 449 450
## [451] 451 452 453 454 455 456 457 458 459 460 461 462 463 464 465 466 467 468
## [469] 469 470 471 472 473 474 475 476 477 478 479 480 481 482 483 484 485 486
## [487] 487 488 489 490 491 492 493 494 495 496 497 498 499 500 501 502 503 504
## [505] 505 506 507 508 509 510 511 512 513 514 515 516 517 518 519 520 521 522
## [523] 523 524 525 526 527 528 529 530 531 532 533 534 535 536 537 538 539 540
## [541] 541 542 543 544 545 546 547 548 549 550 551 552 553 554 555 556 557 558
## [559] 559 560 561 562 563 564 565 566 567 568 569 570 571 572 573 574 575 576
## [577] 577 578 579 580 581 582 583 584 585 586 587 588 589 590 591 592 593 594
## [595] 595 596 597 598 599 600 601 602 603 604 605 606 607 608 609 610 611 612
## [613] 613 614 615 616 617 618 619 620 621 622 623 624 625 626 627 628 629 630
## [631] 631 632 633 634 635 636 637 638 639 640 641 642 643 644 645 646 647 648
## [649] 649 650 651 652 653 654 655 656 657 658 659 660 661 662 663 664 665 666
## [667] 667 668 669 670 671 672 673 674 675 676 677 678 679 680 681 682 683 684
## [685] 685 686 687 688 689 690 691 692 693 694 695 696 697 698 699 700 701 702
## [703] 703 704 705 706 707 708 709 710 711 712 713 714 715 716 717 718 719 720
## [721] 721 722 723 724 725 726 727 728 729 730 731 732 733 734 735 736 737 738
## [739] 739 740 741 742 743 744 745 746 747 748 749 750 751 752 753 754 755 756
## [757] 757 758 759 760 761 762 763 764 765 766 767 768 769 770 771 772 773 774
## [775] 775 776 777 778 779 780 781 782 783 784 785 786 787 788 789 790 791 792
## [793] 793 794 795 796 797 798 799 800 801 802 803
```

Ejercicio 2: Uso de for

Consigna: Calcular la suma de todos los valores del vector *secuencia_dni* usando un for:

```r
Total <- 0
valor_final <- length(secuencia_dni)

for (i in 1:valor_final) {
  Total <- Total + i
}

Total
```

```
## [1] 322806
```

Ejercicio 3: Phyton

Consigna: Repetir el ejercicio anterior pero en Phyton

```python
# --- Ejercicio 1 ---
# Consigna: Crear una variable con las últimas 3 cifras del DNI (803)
dni = 803

# Consigna: Crear un vector (lista) del 1 al 803
# En Python, range(1, 804) genera números del 1 al 803
secuencia_dni = list(range(1, dni + 1))
```

```python
# --- Ejercicio 2 ---
# Calcular la suma de todos los valores usando un bucle 'for'
total = 0

for i in secuencia_dni:
    total += i

# Mostrar el resultado final
print(f"La suma total es: {total}")
```

```
## La suma total es: 322806
```

Ejercicio 4: Medición de tiempo (sys.time)

Consigna:¿Cuanto tarda en correr el código del ejercicio 2?

Usamos el comando **sys.time**

```r
inicio <- Sys.time()
total <- 0
valor_final <- 10000000*length(secuencia_dni)
for (i in 1:valor_final)
total <- total + i
total
```

```
## [1] 3.224045e+19
```

```r
final <- Sys.time()
final-inicio
```

```
## Time difference of 2.2562 mins
```

Ejercicio 5: 2da medición de tiempo (tictoc)

Consigna: Aplicar la consigna anterior pero aprendiendo a usar el comando **tictoc**

```r
library(tictoc)

# Iniciamos el cronómetro asignándole un nombre descriptivo a la prueba
tic("Tiempo de ejecución")

# Inicio del bloque a medir
Total<-0
valor_final<-length(secuencia_dni)

for(i in 1:valor_final) {
Total<-Total + i
}
# Fin del bloque a medir

#Se "detiene" el cronómetro.
```

```r
#Esto imprimirá el tiempo transcurrido en pantalla.
toc()
```

```
## Tiempo de ejecución: 0.004 sec elapsed
```

```r
#Imprimimos la variable Total para verificar que el cálculo matemático se hizo bien
Total
```

```
## [1] 322806
```

Ejercicio 6: Secuencia

Consigna: Generar secuencia de 2 en 2 hasta 50000

```r
inicio_for <- Sys.time()

A <- numeric(50000)
for (i in 1:50000) {
  A[i] <- i * 2
}

final_for <- Sys.time()
tiempo_for <- final_for - inicio_for
```

Secuencia con función de R:

```r
inicio_seq <- Sys.time()

B <- seq(2, 100000, by = 2)

final_seq <- Sys.time()
tiempo_seq <- final_seq - inicio_seq

# Resultados
tiempo_for
```

```
## Time difference of 0.006929874 secs
```

```r
tiempo_seq
```

```
## Time difference of 0.00174284 secs
```

Ejercicio 7: Serie de Fibonacci

Consigna: Generar sucesión o serie de Fibonacci hasta superar 1.000.000

```r
# Generar Fibonacci hasta superar 1.000.000

fibonacci <- c(0,1)
iteraciones <- 2

while (fibonacci[length(fibonacci)] <= 1000000) {
  nuevo <- fibonacci[length(fibonacci)] + fibonacci[length(fibonacci)-1]
  fibonacci <- c(fibonacci, nuevo)
  iteraciones <- iteraciones + 1
```

```
}

fibonacci
```

```
##  [1]       0       1       1       2       3       5       8      13      21
## [10]      34      55      89     144     233     377     610     987    1597
## [19]    2584    4181    6765   10946   17711   28657   46368   75025  121393
## [28]  196418  317811  514229  832040 1346269
```

```
iteraciones
```

```
## [1] 32
```

Ejercicio 8: Definición matemática recurrente

Consigna:¿Cuantas iteraciones se necesitan para generar un número de la serie mayor que 1.000.000?

```
f0<-0   #Valores iniciales
f1<-1
f2<-0
i<-0  #Número de iteraciones incial
valorfinal<-1000000 #Umbral

while (f2<valorfinal) {

  f2=f0+f1
  f0=f1
  f1=f2
  i<-i+1

}
f2
```

```
## [1] 1346269
```

```
i
```

```
## [1] 30
```

Ejercicio 9: Método Burbuja (Bubble Sort)

Consigna: Compara la performance de ordenación del método burbuja vs el método sort de R.

```
x <- sample(1:20000, 20000)

burbuja <- function(x) {
  n <- length(x)
  for (j in 1:(n-1)) {
    for (i in 1:(n-j)) {
      if (x[i] > x[i+1]) {
        temp <- x[i]
        x[i] <- x[i+1]
        x[i+1] <- temp
      }
    }
```

```r
  }
  return(x)
}
```

Comparación:

```r
system.time({
  res1 <- burbuja(x)
})
```

```
##    user  system elapsed
##  20.589   0.004  20.681
```

```r
system.time({
  res2 <- sort(x)
})
```

```
##    user  system elapsed
##   0.001   0.000   0.000
```

Ejercicio 10: Penitencia de Newton

Consigna: Desarrollar dos algoritmos que hagan el trabajo de sumar desde 1 hasta 1000000 y verifcar cuál de los dos es más eficiente

Método 1 (for):

```r
n <- 1000000

system.time({
  suma1 <- 0
  for (i in 1:n) {
    suma1 <- suma1 + i
  }
})
```

```
##    user  system elapsed
##   0.019   0.000   0.018
```

Método 2 (fórmula matemática):

```r
system.time({
  suma2 <- n * (n + 1) / 2
})
```

```
##    user  system elapsed
##       0       0       0
```

Comparación:

```r
suma1
```

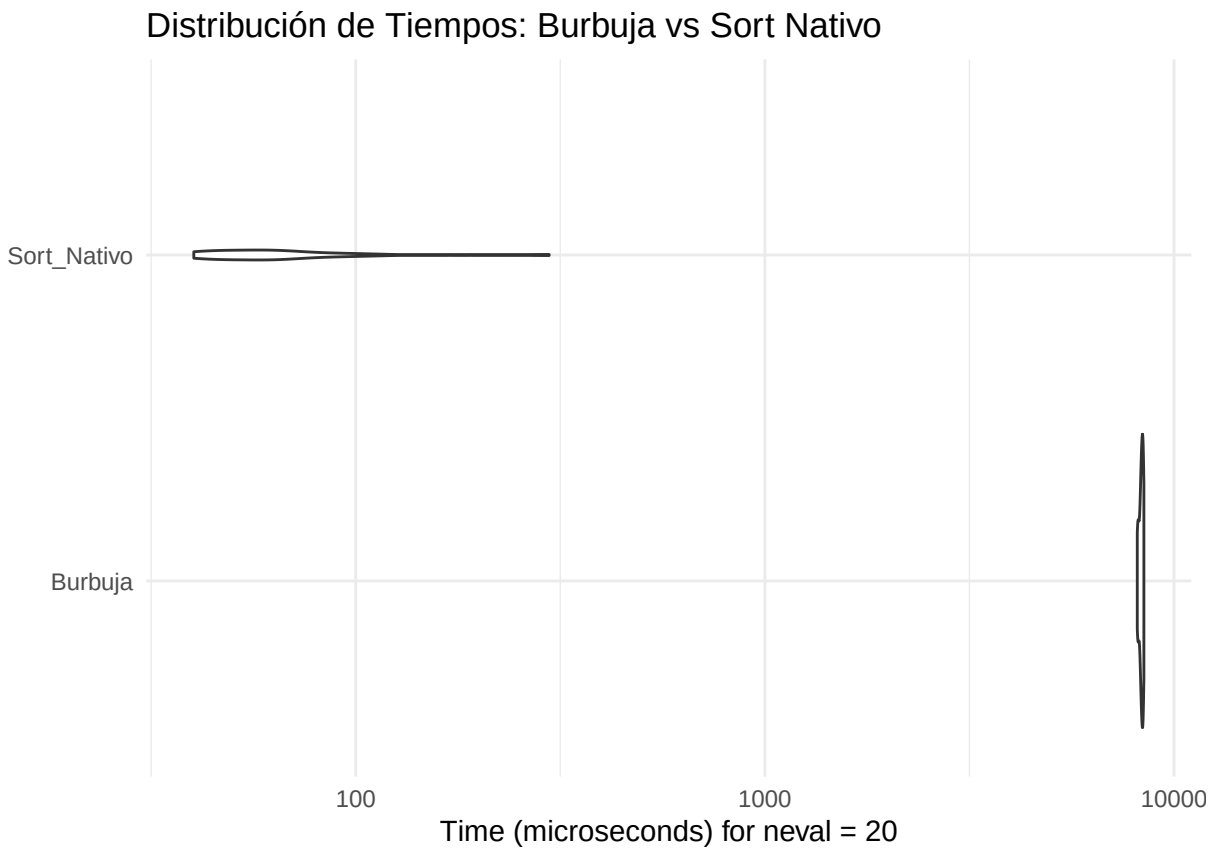
```
## [1] 500000500000
```

```r
suma2
```

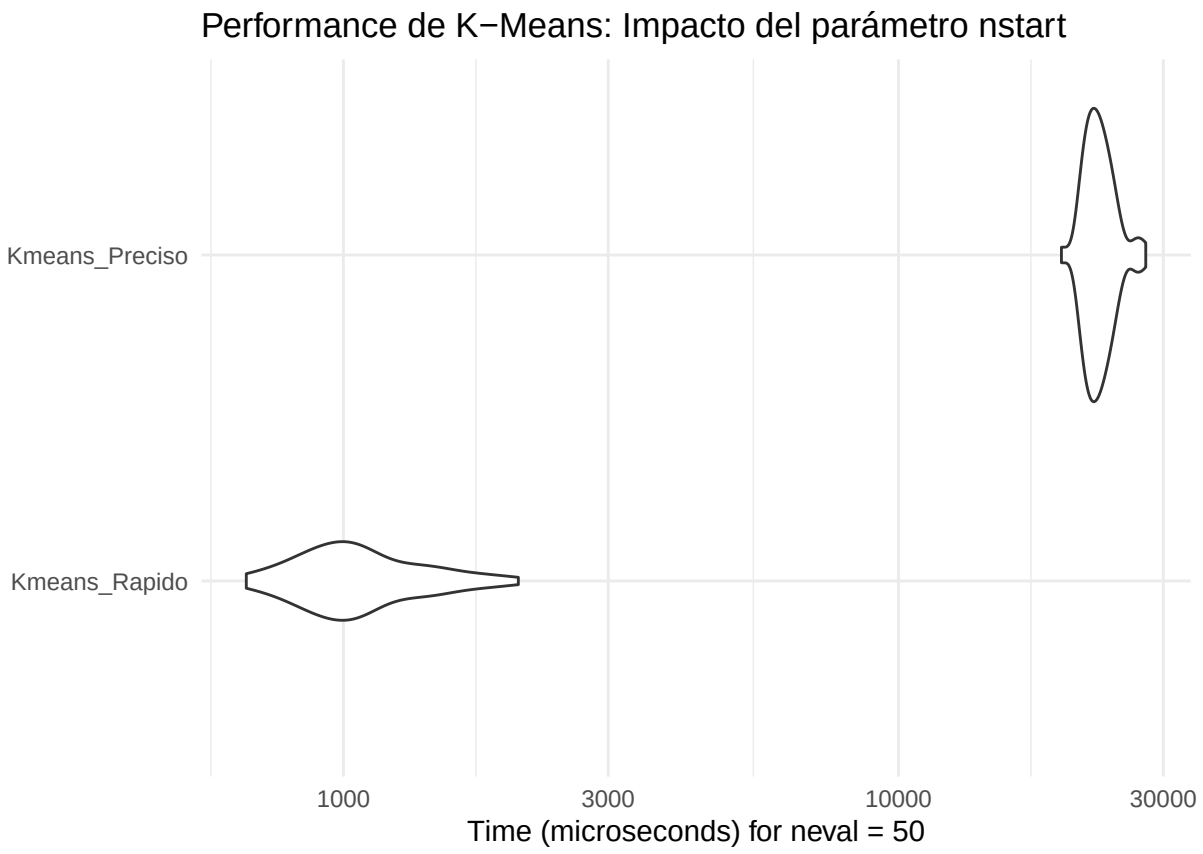
```
## [1] 500000500000
```

Análisis de Rendimiento: Ordenamiento y Agrupación

```r
library(microbenchmark)
library(ggplot2)

# Definimos la función del método burbuja (del apunte original)
burbuja <- function(x){
  n <- length(x)
  for(j in 1:(n-1)) {
    for(i in 1:(n-j)){
      if(x[i] > x[i+1]){
        temp <- x[i]
        x[i] <- x[i+1]
        x[i+1] <- temp
      }
    }
  }
  return(x)
}

# Generamos una muestra aleatoria moderada para la prueba
set.seed(123)
muestra <- sample(1:1000, 400, replace = TRUE)

# Ejecutamos el benchmarking
mbm_orden <- microbenchmark(
  Burbuja = burbuja(muestra),
  Sort_Nativo = sort(muestra),
  times = 20 # Repetimos 20 veces para tener una buena distribución
)

# Visualizamos con gráfico de violín
autoplot(mbm_orden) +
  ggtitle("Distribución de Tiempos: Burbuja vs Sort Nativo") +
  theme_minimal()
```




```r
# Generamos una matriz de datos simulada (ej: variables de mantenimiento industrial)
set.seed(42)
datos_industriales <- matrix(rnorm(5000), ncol = 5)

# Benchmarking del algoritmo K-Means
mbm_kmeans <- microbenchmark(
  Kmeans_Rapido = kmeans(datos_industriales, centers = 4, nstart = 1),
  Kmeans_Preciso = kmeans(datos_industriales, centers = 4, nstart = 25),
  times = 50
)

# Visualizamos con gráfico de violín
autoplot(mbm_kmeans) +
  ggtitle("Performance de K-Means: Impacto del parámetro nstart") +
  theme_minimal()
```



Conclusión final

A lo largo de este trabajo, evaluamos la eficiencia de distintos algoritmos en R mediante técnicas de benchmarking (como Sys.time, tictoc y microbenchmark). Comprobamos empíricamente que la calidad de un código no radica solo en llegar al resultado correcto, sino en cómo optimiza el tiempo y los recursos del sistema.

La incorporación de los gráficos de violín resultó fundamental para visualizar estadísticamente no solo la media de ejecución, sino la estabilidad y distribución temporal de procesos analíticos complejos, como la comparación de métodos de ordenamiento y la medición de performance del agrupamiento K-Means. Es fundamental destacar que los tiempos absolutos medidos y la forma exacta de la "panza" de los violines serán diferentes en cada computadora; estos resultados variarán dinámicamente según las propiedades físicas y la capacidad de procesamiento (hardware) del equipo donde se compile el documento.

Sin embargo, más allá de la máquina específica que se utilice, el principio de optimización estructural se mantiene inalterable. El ejercicio de "La penitencia de Newton" ilustró perfectamente este concepto: mientras la fuerza bruta de un bucle iterativo demanda un costo computacional inmenso para realizar operaciones masivas, la aplicación de un modelo matemático sólido resuelve el problema en fracciones de segundo.

En definitiva, la evaluación de estos algoritmos nos confirma que la herramienta de optimización más poderosa en la programación moderna no es la computadora que tengamos, sino el razonamiento lógico, físico y matemático que aplicamos antes de escribir la primera línea de código.